Chladni Plate Inverse Design System: Five Algorithmic Improvement Directions

(with a Review of the Trial History)

Chladni Studio Project Team

June 2026

Abstract

What our software does can be summed up in one sentence: the user draws a pattern, and the program works backwards to design an unevenly thick plate and a few drive frequencies so that sand on the vibrating plate arranges itself into that pattern as closely as possible. After 350+ experiments, the current pipeline can make the sand on the target lines 4.85 times denser than the plate average for the "IC" letters target, on a printable carbon-fibre plate (CF-PETG) — close to the limit of this physical setup. This article explains, in plain language wherever possible, five directions in which the system can still improve: (1) a scoring rework, so the score actually reflects "does it look right"; (2) geometry-aware scoring, so the program knows where the sand is misplaced and which way to move it; (3) a frequency inner loop, so each plate is first given its best frequency and only then judged; (4) randomised restarts, so an unsatisfying result triggers a fresh attempt from a new starting point; and (5) fixing the centre artefact, a modelling error that piles phantom sand at the plate centre in every simulation. The article closes with a review of the key turning points in the project's trial history, correcting a few inaccurate explanations that have been circulating.

1 Background: How the System Works Today

The pipeline has four steps. First, the user supplies a 256×256 black-and-white target image. Second, a fast "draft physics engine" that we wrote ourselves (a *surrogate* model) iterates about 300 times, jointly adjusting the plate's 15×15 thickness layout H and six candidate drive frequencies so that the predicted sand picture approaches the target. Third, the plate is handed to the accurate-but-slow COMSOL simulation, which computes its first 30 resonance modes; the frequencies that contribute most to the target are then selected (this is Phase 1), plus one "lucky frequency" (165 Hz) found by manual sweeping. Fourth (Phase 2), the COMSOL results are fed back to fine-tune the plate in small, careful steps — keep what improves, roll back what doesn't — for one round.

Equally important are the **physical boundaries** — some of the "doesn't look right" is not the algorithm's fault:

- **Single centre excitation**: the shaker pushes the plate at one central point only, so the modes it can effectively drive are biased towards "the most symmetric family" (the A1 representation, in group-theory terms).

- **Square symmetry** (D4): a square has eight rotations and reflections that map it onto itself. When both plate and excitation are symmetric, the resulting patterns strongly tend to be symmetric too, so asymmetric targets (like the letters IC) start at a disadvantage.
- **Resolution**: only 15×15 thickness cells are adjustable, and the printable anisotropy (stiffness ratio) tops out around 3.

Within these constraints, 4.85× enrichment is close to the practical ceiling. The five directions below do not try to beat the physics; they make the scoring more honest, the search smarter, and the simulation closer to how real sand behaves.

Glossary

Term	Plain-language meaning
surrogate	our hand-written fast approximate physics engine: rough but quick, used instead of slow COMSOL for the many iterations
enrichment (E)	how many times denser the sand is on the target lines than the plate average; higher is better
recall (R)	what fraction of the target lines actually falls inside the "quiet zone" (the low-vibration region where sand settles)
contrast (C)	sand density in the target region versus the background
nodal line / quiet zone	where the plate barely vibrates; sand ends up there
D4 / A1	the square's eight self-mapping operations / the most symmetric family of vibration shapes, unchanged by all of them
trust region	"small careful steps": enlarge the step after an improvement, shrink and roll back after a setback
off-resonance / lucky frequency	a frequency not sitting on a resonance peak that nevertheless helps the target a lot (e.g. 165 Hz)
PCA design manifold	a few dozen "directions of useful change" distilled from historically good designs
optimal transport (OT)	the minimum effort needed to "carry" the current sand picture into the target one; the carrying plan says which pile moves where

2 Direction 1: Scoring-System Rework

2.1 Current state and problem

How does the program know whether a plate is "good"? By a score. The optimisation loss currently used is

$$\mathcal{L} \;=\; -\log E \;-\; \log C \;-\; 0.5\,\log R \;\;\equiv\;\; -\log\!\big(E \cdot C \cdot R^{0.5}\big), \tag{1}$$

essentially **multiplying** enrichment, contrast, and recall together. Ranking uses a separate weighted sum,

$$S_{\text{comp}} = 0.40\,\text{clip}\big(\log(1+\max(E-1,0)),0,2\big)+0.30\,R+0.20\,\text{clip}\big(\log(1+\max(C-1,0)),0,2\big)+0.10\,D, \tag{2}$$

where D measures whether the sand picture's main axis agrees with the target's.

The observed problem: **a high score does not mean it looks right**. The classic case: the target was a diagonal line, one plate scored a healthy 2.41× enrichment — and the actual pattern was an X. The reason is easy to see: enrichment only asks "is the sand dense on the target line", not "does the sand cover the whole line". Concentrating sand heavily on **one short segment** of the target inflates the score even when the overall picture looks nothing like the target.

2.2 Proposed rework

Change of mindset: enrichment and contrast stop competing in the ranking and become **pass marks** — manually set floors below which a candidate is simply discarded. Among the candidates that pass, rank by "how much it looks like the target".

But "looks like" cannot be recall alone. Recall has a blind spot: it only asks "how much of the target is covered" and ignores spurious lines **outside** the target — and spurious lines are usually exactly why a pattern looks wrong. The ranking key should therefore combine two questions: "how much of the target is covered" (recall) and "how much of the quiet zone actually lies on the target" (precision). Their standard combination is the F_β score:

$$S_{\text{rank}} = F_\beta(\text{valley, target}) \quad \text{s.t.} \quad E \geq \tau_E,\ C \geq \tau_C. \tag{3}$$

Three implementation details: (a) the optimiser needs everything differentiable, and a hard pass mark is not; replace it with a hinge penalty (the further below the mark, the heavier the penalty; zero once passed); (b) fix a known small bug — a degenerate all-zero vibration field is currently graded $R = 1.0$ (full marks) and should score zero; (c) the pass marks must be calibrated to the target's line thickness and area, not fixed globally.

3 Direction 2: Geometry-Aware Scoring

3.1 Motivation

All three current metrics are "position-blind": they can say *how much* sand landed on the target, but not *where* the sand is misplaced or *which way* it should move. Two patterns with identical scores can be geometrically miles apart. It is like telling a student they scored 60 without showing which answers were wrong — no sense of direction. Making the algorithm "understand geometric trends like a human" has rigorous mathematical counterparts, and **two of them already have skeletons in the codebase**:

3.2 Proposed additions

1. **Distance-field loss** (skeleton in `src/scoring/signed_distance_loss.py`): bill every grain of sand for "how far it sits from the target line",

$$\mathcal{L}_{\text{SDF}} = \frac{\sum_x p(x)\, d_T(x)}{\sum_x p(x)}, \tag{4}$$

where p is the sand density and d_T the distance to the target. The further away the sand, the heavier the penalty — so the gradient naturally pulls sand toward the target.

2. **Optimal-transport loss** (skeleton in `src/physics/sinkhorn_loss.py`): compute the minimum cost of "carrying" the current sand picture into the target one. The transport plan literally is the requested "direction of progress" — which pile moves where, and how far.

3. **Multi-scale comparison**: score on several increasingly blurred versions simultaneously — get the coarse shape right first, then the details. This matches how human eyes judge, and it also smooths the optimisation landscape.

4. **Topological sanity checks** (final ranking only): count connected components and skeleton branches. Enrichment cannot tell a diagonal from an X; counting branches can.

One historical concern must be addressed: the project tried pixel-wise comparisons (IoU / Dice) early on and abandoned them. But those failed because they compared **discrete nodal-line masks** (non-differentiable; scores jumped whenever modes switched). Today's sand-density field is continuous and differentiable; adding distance-field / optimal-transport losses on top of it does not repeat that mistake. Division of labour: the new losses **point the way** (used in optimisation), while enrichment / recall remain the **acceptance and ranking metrics**. Complementary, not a replacement.

4 Direction 3: Frequency Inner Loop in the Gradient Logic

4.1 Current state and problem

The optimiser currently descends on thickness and frequencies **at the same time**. The trouble is that the plate's resonance peaks are entirely determined by the thickness layout: change the thickness and all the peaks move, while the frequency variables can only crawl after them along local gradients. The consequence: **a genuinely good plate can get a poor score merely because the current frequencies don't suit it, and be vetoed by the descent direction**. An analogy: judging a guitar without allowing it to be tuned first — good instruments get wronged.

The existing mitigations (frequencies frozen for the first 60 steps, six frequencies at once, a spacing penalty, etc.) treat the symptoms. Notably, the pipeline already does the right thing at the COMSOL stage — Phase 1 is precisely "look at all the modes of this plate first, then pick frequencies" — but only **once, after** the optimisation has finished.

4.2 Proposed rework

Move "tune first, judge second" inside the optimisation loop:

$$S(H) \;=\; \max_{f \,\in\, [f_{\min},\, f_{\max}]} s(H, f), \tag{5}$$

i.e. at every step, sweep the whole frequency range for the current plate, find its best frequency, score there, and only then take the gradient with respect to thickness. Mathematics backs this up (Danskin's theorem): the gradient computed at the optimal frequency is the correct gradient.

The worry that "sweeping the whole range is too slow" has an answer: **modal superposition**. Solve the plate's vibration modes once per step (one eigendecomposition); after that, the response at any number of frequencies can be assembled from those modes almost for free — possibly faster than today's one-direct-solve-per-frequency scheme. Two details: (a) the best frequency jumps abruptly when two modes swap places; the hard max is discontinuous there, so use a smooth "soft-max" instead; (b) the team's intuition that "steps must be small enough that the new plate's best frequency stays close to the old plate's" is exactly right — that is the trust-region idea, and it can be merged with Phase 2's existing keep-or-roll-back logic. Also, "start from a flat plate" is already the status quo (thickness initialises as a uniform plate), and the "is this target even achievable" pre-check already exists (`target_realizability.py`); both carry over directly.

5 Direction 4: Result Evaluation and Randomised Restarts

5.1 Current state and problem

Gradient descent is a "walk downhill" algorithm: it stops at the bottom of whatever valley it is in, never knowing whether the next valley over is deeper (the local-optimum problem). The project once built multi-start infrastructure (descend from N starting points, keep the best), but after the θ optimisation was removed the optimiser became **deterministic** — the same start gives the identical result every single time, so multi-start now exists in name only (the code marks it as a no-op).

5.2 Proposed rework

1. **Restore randomness first**: reintroduce random thickness initialisation (or controlled perturbations of the uniform start). Without randomness, "restart" is an empty word.

2. **Restart in a smarter space**: rather than scattering points over the raw 225 thickness cells, sample inside the "PCA design manifold" — the few dozen directions of useful change distilled from historically good designs. "Far enough from the original start" is guaranteed by standard spread-out sampling (Latin hypercube, maximin).

3. **Trigger restarts by a pass mark — but not an arbitrary one**: IC's 4.85× is already near the physical ceiling; set the bar above the ceiling and the program restarts forever. Derive the bar from the "how much is this target theoretically achievable" analysis, adapt it per target, and cap the number of restarts.

4. **Reuse the existing anti-regression contract**: the "new answer must not be worse than the old one; dethroning the champion requires an above-threshold improvement" logic carries over as-is.

This direction should come after Directions 1 and 2 — both "how do we decide we're unsatisfied" and "how do we compare candidates" depend on a more trustworthy score.

6 Direction 5: Centre-Excitation Artefact Correction

6.1 Problem confirmation

Experiments show a stubborn contradiction between simulation and reality: **in reality, sand almost never stays at the excitation point (the plate centre) — it is the most violently shaken spot on the plate, and sand is thrown off first; yet every simulation output piles sand exactly there.**

A code review confirms a systematic modelling error, with the following mechanism. The simulation describes base excitation in a "relative frame": the centre clamp's displacement is fixed at zero (the centre degrees of freedom are removed in `orthotropic_plate.py`, and `expand_to_grid` fills the centre cells with zeros) — equivalent to watching the plate while standing on the shaker. In that frame the centre's **relative** displacement is identically zero. Meanwhile the powder model

$$p(x) = \exp\Big(-\big(|u(x)|/\sigma\big)^2\Big) \tag{6}$$

follows the rule "sand piles up wherever the vibration is small" — so the centre is misjudged as the quietest spot and grows a pile of **phantom sand**. Real sand does not listen to relative displacement; it listens to **absolute acceleration**: sand stays only where the absolute acceleration is below a critical value (of the order of gravity g) — and the centre's absolute acceleration is the largest on the whole plate.

6.2 Proposed fixes

1. **Quick fix (one line of code)**: simply exclude the centre-clamp region from scoring. Apply it on both sides — the surrogate and the COMSOL scorer (`score_comsol_response.py`).
2. **Proper fix (physically correct)**: score on absolute displacement. Under harmonic excitation the base displacement amplitude is a_{base}/ω^2, so

   $$u_{\text{abs}}(x) = u_{\text{rel}}(x) + a_{\text{base}}/\omega^2; \tag{7}$$

   with this term included the centre is no longer zero, and the phantom pile vanishes by itself.
3. **Powder-model upgrade**: replace "sand where relative displacement is small" by a soft switch "sand where absolute acceleration is below threshold", e.g. $p \propto \sigma\big(\beta\,(a_{\text{crit}} - \omega^2|u_{\text{abs}}|)\big)$ — which is the real sand physics.

6.3 Knock-on consequences

The phantom sand is not a cosmetic flaw: it entered the total sand budget (the denominator of enrichment), distorted the "quiet zone" threshold, and depressed contrast — **every historical score has been contaminated by it**. After the fix, the 4.85× baseline will certainly move (most likely upward for targets that avoid the centre), and the lucky frequency 165 Hz and the Phase 1 mode choices may need re-sweeping. This direction should therefore be done **first**: it is the cheapest and benefits everything. Conversely, if it is not fixed first, the optimal-transport loss of Direction 2 will dutifully "carry" the fake sand pile onto the target — giving the wrong direction altogether.

7 A Review of the Trial History, with Corrections

The team's trial notes ("trails") preserve many valuable judgements, but in a few places the recorded explanation of *why* a change was made differs from the evidence stored in the repository. This section connects the trial history to the five directions above and corrects the record along the way.

7.1 The materials journey: uniform → anisotropic → dropping θ

The storyline in the notes is right: the project first used SLA printing to maximise material uniformity, then found that **even plates with large thickness differences kept producing stubbornly symmetric patterns barely different from a uniform plate**; it switched to FDM-printed anisotropic material; it first gave every small cell its own fibre orientation (the θ field), found that this performed poorly, and fell back to one orientation for the whole plate.

The physics behind this path can be stated more sharply. Thickness alone is a weak symmetry breaker because bending stiffness scales with the **cube** of thickness, yet the printable thickness range is limited — there is only so much leverage. A uniform anisotropy direction, by contrast, immediately reduces the square's eight-fold symmetry (D4) to four-fold (D2) — which is why the patterns could depart from the uniform plate yet still keep mirror symmetries. That matches the team's observation exactly: "different from uniform, but still quite symmetric".

What needs correcting is **the reason for dropping θ**. The notes say "too many degrees of freedom confused the program and slowed it down". The mechanism recorded in the repository (the W10 module documentation, citing the controlled experiments of May 2026) is different: W10 actually operates in the **off-resonance** regime, where the inertia term exceeds the stiffness term by roughly five orders of magnitude ($\omega^2 M \gg K$), and θ enters the equations only through stiffness — so **the gradient of θ is nearly zero**. θ was not "so numerous it confused the program"; it was a **parasitic variable that physics barely lets act**. The optimiser could not push it, so it drifted randomly, and the drifted θ then degraded the anisotropic COMSOL modes downstream. A ten-run controlled battery (5 targets × 2 materials) confirmed it: isotropic PLA actually **beat** carbon-fibre plates with "optimised" θ on 4 of 6 targets. This version of the causal chain is more accurate — and more persuasive.

A documentation issue follows: `docs/final_results.zh-CN.md` still claims "key insight: $\theta \neq 0$ breaks D4 symmetry", contradicting the current code (θ held at zero). It should be revised.

7.2 The scoring journey: re-attributing "sand only piles in the middle"

The notes record: the earliest scoring rewarded sand near the target and punished sand away from it; the program then kept producing designs whose sand "only accumulated in the middle"; this was attributed to physical limitations ("the differential equations simply have no solution shaped like our target"), and the metrics were replaced by enrichment / recall / contrast.

That attribution is only half right. In the light of Direction 5, **a good part of the early "middle piling" was very likely the centre phantom-sand artefact**: the modelling error marks the centre as the best place for sand, so the program naturally put its "output" there.

The physical limitation is real, but it shows up as "patterns lean symmetric and simple", not as "all the sand crowds into the centre". We recommend adding this re-attribution to the notes — it ties two independent puzzles (the early scoring oddity and today's centre pile) to a single root cause.

Also worth adding as a prequel: before the three metrics there was a generation of pixel-wise IoU / Dice scoring (Phases A–D in the history), which failed because it was discrete, non-differentiable, and jumped whenever modes switched. What saved the scoring system was the **powder-physics reformulation** (sand settles in amplitude valleys). That history is also why Direction 2 dares to reintroduce "geometric comparison" — this time on the continuous, differentiable sand-density field, not on discrete masks.

7.3 What the surrogate actually is

The notes describe the surrogate as "a differential equation solving tool in PyTorch". More accurately: it is **not an off-the-shelf tool** but our own plate-vibration solver (the Kirchhoff–Love plate equation discretised on a coarse grid). The decisive reason for writing it in PyTorch is not speed but PyTorch's **automatic differentiation** — it makes "the derivative of the sand picture with respect to the thickness" directly computable, which is what makes the whole gradient-based design route possible. The speed comes from three choices: a coarse grid (25×25), giving up high accuracy, and not chasing agreement with COMSOL — only the direction needs to be right. COMSOL remains the final judge; the surrogate is the scratch paper on which the answer is drafted.

7.4 The off-resonance decision and its chain reaction

"Escaping the eigenmodes" (using any frequency that yields a stable pattern, not just resonances) was a good decision supported by the metal-plate experiments, and the lucky frequency 165 Hz is its direct product. But one chain reaction went unnoticed at the time: **it was precisely the move into the off-resonance regime that turned θ into a parasitic variable** (off resonance, $\omega^2 M \gg K$; see Section 8.1). Two seemingly independent trail entries are in fact one causal chain: first the project chose off-resonance operation, then it discovered θ was useless — the latter is a necessary consequence of the former. Writing this chain down turns the trial history from a logbook into an explanatory narrative.

8 Implementation Roadmap and Expected Payoff

9 Conclusion

The five directions target three weak links of one main line: **honesty of the score** (Directions 1, 2, 5), **efficiency of the search** (Directions 3, 4), and **fidelity of the simulation** (Direction 5). None of them hopes to break the physical ceiling set by single-point excitation and square symmetry; they make sure the algorithm wastes none of the room physics allows. The trial-history review adds a methodological reminder: **keep the real evidence for "why we changed it" in the record** (the vanishing gradient of θ, the modelling root cause of the centre

Stage	Direction	Effort	Expected payoff
0	5: centre-artefact fix (mask, then absolute response)	Small	Debias all metrics; re-baseli
1	1: scoring rework (pass marks + “looks-like” ranking)	Small–medium	Scores agree with the eye
1	2: geometric losses (distance field / OT + multi-scale)	Medium	The program knows where s
2	3: frequency inner loop (tune first, judge second)	Medium–large	Good plates no longer wron
3	4: randomised restarts (smarter starting points)	Medium	Escape local optima

Table 1: Roadmap. Stage 0 is a prerequisite: without clearing the phantom sand first, the new score and the geometric losses would both be working on contaminated data.

pile) — it matters far more than recording only “what we changed”. Several of this project’s most valuable advances came precisely from re-attributing old conclusions. As for order of execution: do the cheap, globally beneficial corrections first (centre artefact, ranking key), and restructure the optimiser afterwards (frequency inner loop, restart strategy) — that is the lowest-risk path.